

MiDAS Feature Breakdown Agents – Developer Guide

This guide explains how to use the **MiDAS Feature Breakdown Agents** in VSCode Copilot Chat to automate feature planning and story creation.

Introduction

Two agents help streamline your GitHub workflow:

1. **Create Feature Agent** – Creates structured GitHub feature issues
 2. **Feature to Story Agent** – Breaks features into actionable development stories
-

Prerequisites

- The agent requires an authenticated GitHub MCP connection with permissions to create issues, add labels, and assign users.
 - If no valid connection is configured, the agent will prompt to connect and issue creation will fail until a connection with appropriate scopes is provided.
-

1. Create Feature Agent

What It Does

Creates comprehensive GitHub feature issues with proper structure, acceptance criteria, and documentation links.



When to Use

- Creating a new feature request
- Documenting feature requirements
- Ensuring consistent feature formatting

How to Use

Step 1: Prepare Input File

Create an `inputs.md` file with your feature details:

```
# Feature Request Inputs

## Required Fields

### Feature Description
<!-- Provide a concise, value-driven description of the feature to be
created -->

### Repository Context
**Repository Owner:** <!-- Default: Midas -->
**Repository Name:** <!-- Default: Midas-Platform -->

### Documentation Context
<!-- Provide one of the following: - Documentation folder path (e.g.,
/docs, /design, /architecture) - Any relevant path or link for context -
"No documentation available" if none exists -->

### Feature Template Path
<!-- Provide the path to the Feature Issue Template if one exists: -
Template path (e.g., /docs/Feature_Template.md,
.github/ISSUE_TEMPLATE/feature.md) - "No template available" if none
exists -->

## Optional Fields

### Additional Labels
<!-- List any additional labels besides "Feature" (comma-separated) -->

### Reviewers
<!-- List GitHub usernames of reviewers to assign (comma-separated) -->

### User Tags
<!-- List GitHub usernames to tag in the issue (comma-separated) -->
```



```
# Feature Request Inputs

This file contains all the necessary inputs for creating a Github feature request issue.

---
## Required Fields

### Feature Description
Currently, onboarding a new use case requires manual updates to API backend logic, database entries, and sub-phase configuration.
This creates dependency on the platform team and slows down the onboarding process.

This feature proposes introducing dedicated API endpoints that allow authorized use case owners to onboard new use cases independently through configuration-driven workflows.

The goal is to:
- Eliminate manual backend and database changes
- Remove dependency on platform intervention
- Enable onboarding independent of build/release cycles
- Restrict configuration access to G8Admin and Power users

---
## Repository Context

**Repository Owner:**
MiDAS

**Repository Name:**
MiDAS-Platform

---
## Documentation Context
MiDAS-Platform\docs

---
### Feature Template Path
MiDAS-Platform\docs\Feature_Template.md

---
## Optional Fields

### Additional Labels
UseCase, 10-R1

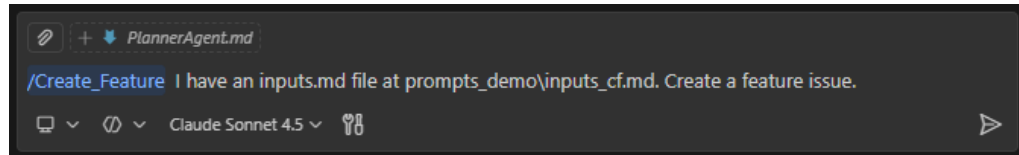
---
### Reviewers
<!-- None specified -->

---
### User Tags
<!-- None specified -->
```

Note: If no inputs.md is provided, the agent starts an interactive input flow to collect required fields. Missing required fields are prompted for and block analysis/creation until provided or explicitly skipped (skipping may reduce quality); optional fields are requested but can be skipped. The agent always summarizes collected inputs and requires an explicit "Yes" before creating any issues.

Step 2: Trigger in VSCode Copilot Chat

```
/Create_Feature I have an inputs.md file at /path/to/inputs.md. Create a feature issue.
```



Step 3: Review and Confirm

The agent will validate inputs and ask for confirmation before creating the issue.



I'll help you create a feature issue by following the Feature Request Architect workflow. Let me start by reading the inputs file you've provided.

✓ Reviewed inputs_cf.md

Great! I've successfully read the inputs file. Let me validate the inputs and then proceed:

Input Validation Summary:

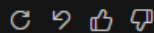
Required Fields:

- ✓ Feature Description: Self-service use case onboarding API
- ✓ Repository Owner: MiDAS
- ✓ Repository Name: MiDAS-Platform
- ✓ Documentation Context: MiDAS-Platform\docs
- ✓ Feature Template Path: MiDAS-Platform\docs\Feature_Template.md

Optional Fields:

- Additional Labels: UseCase, 10-R1
- Reviewers: None specified
- User Tags: None specified

All required fields are present. Should I proceed with creating the feature issue? (Please confirm with "Yes")



🔧 Run Create or update issue. – github (MCP Server)



Create a new or update an existing issue in a GitHub repository.

Input

```
{
  "method": "create",
  "owner": "MiDAS",
  "repo": "MiDAS-Platform",
  "title": "Self-Service Use Case Onboarding via Configuration-Driven APIs",
  "labels": [
    "Feature",
    "UseCase",
    "10-R1"
  ],
  "body": "## Description\nThis feature enables the MiDAS platform to provide self-service use case onboarding through configuration-driven API endpoints, eliminating manual backend changes and database interventions."
```

Note that MCP servers or malicious conversation content may attempt to misuse 'Code' through tools.

Allow

Skip



✔ Feature issue created successfully!

Issue Details:

- **Title:** Self-Service Use Case Onboarding via Configuration-Driven APIs
- **Issue Number:** #3786
- **Repository:** MiDAS/MiDAS-Platform
- **Labels:** Feature, UseCase, 10-R1

What was created:

The feature issue includes:

- **Description:** Clear explanation of the self-service onboarding capability to eliminate manual backend changes
- **Scope:** 8 comprehensive bullet points covering API endpoints, RBAC integration, configuration schemas, sub-phase management, database integration, backward compatibility, documentation, and audit/monitoring
- **Acceptance Criteria:** 14 measurable and testable criteria including API endpoints, authentication, validation, database integration, Git workflows, admin interface, audit logging, documentation, error handling, backward compatibility, code coverage, and diagrams
- **Related Documentation:** Links to 5 relevant existing documentation files for context
- **Implementation Notes:** 5 practical guidance points for developers

The feature was structured using the existing Feature Template and informed by comprehensive analysis of:

- Existing use case onboarding API in SS3 and SS6
- Platform configuration management system
- Role-based authorization (GBAdmin and Power users)
- API endpoint structure and patterns
- Database schema design

The feature is ready for developer review and implementation planning! 🚀



Step 4: Feature Created

A new GitHub feature issue is created with proper structure and labels.



The screenshot shows a GitHub issue page for 'Self-Service Use Case Onboarding via Configuration-Driven APIs #3786'. The issue is open, has 14 tasks, and was opened 2 minutes ago by KES1HYD. The description explains that this feature enables the MIDAS platform to provide self-service use case onboarding through configuration-driven API endpoints, eliminating manual backend changes and database interventions. It details the introduction of dedicated API endpoints for authorized users (GBAdmin and Power users) to independently onboard new use cases through configuration-driven workflows. The scope includes API endpoints for self-service configuration, role-based access control integration, configuration schema and validation, sub-phase configuration management, database configuration integration, backward compatibility, API documentation and examples, and audit and monitoring. The acceptance criteria list that API endpoints are implemented in S56, authentication enforces role-based access control, and JSON schema validation is implemented for use case configurations.

Description

This feature enables the MIDAS platform to provide self-service use case onboarding through configuration-driven API endpoints, eliminating manual backend changes and database interventions. Currently, onboarding a new use case requires manual updates to API backend logic, database entries, and sub-phase configuration, which creates dependency on the platform team and slows down the onboarding process.

By introducing dedicated API endpoints that allow authorized use case owners (GBAdmin and Power users) to independently onboard new use cases through configuration-driven workflows, the platform will eliminate manual backend and database changes, remove dependency on platform intervention, enable onboarding independent of build/release cycles, and restrict configuration access to authorized roles only.

Scope

- **API Endpoints for Self-Service Configuration:** Design and implement RESTful API endpoints enabling GBAdmin and Power users to create, update, and manage use case configurations including sub-phase definitions, metadata schemas, and workflow parameters without backend code modifications
- **Role-Based Access Control Integration:** Leverage the existing SS12 authorization system to enforce permissions ensuring only GBAdmin and Power users can access use case onboarding and configuration endpoints
- **Configuration Schema and Validation:** Define comprehensive JSON schemas for use case configurations stored in the `config_schema` database tables with runtime validation to ensure data consistency and prevent invalid configurations
- **Sub-Phase Configuration Management:** Extend the existing use case onboarding API to support dynamic sub-phase configuration through the API, allowing authorized users to define phases, sub-phases, and their relationships without database access
- **Database Configuration Integration:** Build upon the existing platform configuration management system documented in `PLATFORM_CONFIGURATION_DESIGN.md` to store use case onboarding configurations with full audit history
- **Backward Compatibility:** Ensure the enhanced self-service APIs work seamlessly with the existing `/services/api/v1/usecase-onboard` endpoint and maintain compatibility with current use case definitions in the `UC_METADATA` table
- **API Documentation and Examples:** Update API endpoint documentation in `api_endpoints.md` and provide comprehensive examples for GBAdmin and Power users demonstrating configuration workflows
- **Audit and Monitoring:** Implement logging and audit trails for all configuration changes using existing audit patterns to track who made changes, when, and what was modified

Acceptance Criteria

- **API Endpoints:** Self-service use case configuration endpoints are implemented in S56 (`serviceapi-platform`) and accessible at `/services/api/v1/use-case-config/*` supporting CREATE, READ, UPDATE operations
- **Authentication:** API endpoints enforce role-based access control via SS12 authorization system, restricting access to GBAdmin and Power users only with appropriate 403 responses for unauthorized access attempts
- **Configuration Schema:** JSON schema validation is implemented for use case configurations stored in

Assignees
No one—assign yourself

Labels
10-R1 Feature UseCase

Projects
MIDAS Platform Management
Status: No status +9 more

Milestone
No milestone

Development
Create a branch for this issue or link a pull request.

Notifications
Unsubscribe
You're receiving notifications because you authored the thread.

1 participant

Lock conversation
Pin issue
Transfer issue
Convert to discussion

2. Feature to Story Agent

What It Does

Breaks down feature issues into multiple development stories with clear tasks and acceptance criteria.

When to Use

- Feature needs implementation planning
- Breaking work into smaller deliverables
- Creating developer-ready tasks

How to Use



Step 1: Prepare Input File

Create an `inputs.md` file with the feature reference:

```
# Story Breakdown Inputs

## Required Fields

### Feature Issue Reference
<!-- Provide the GitHub feature issue URL or issue number:
- Full URL: https://github.com/owner/repo/issues/123
- Issue number: #123 (will use default repository)
-->

### Repository Context
**Repository Owner:** <!-- Default: Midas -->
**Repository Name:** <!-- Default: Midas-Platform -->

## Optional Fields

### Impact Points
<!-- List specific areas of focus or implementation priorities (comma-separated or line-separated):
Examples:
- authentication module
- user interface
- API endpoints
- database schema
-->

### Additional Labels
<!-- List any additional labels besides "Story" (comma-separated) -->

### Assignees
<!-- List GitHub usernames of assignees (comma-separated) -->
```



```
# Story Breakdown Inputs

This file contains all the necessary inputs for breaking down a feature into development stories.

---

## Required Fields

### Feature Issue Reference
https://github.boschdevcloud.com/MiDAS/MiDAS-Platform/issues/3786

### Repository Context

**Repository Owner:**
MiDAS

**Repository Name:**
MiDAS-Platform

---

## Optional Fields

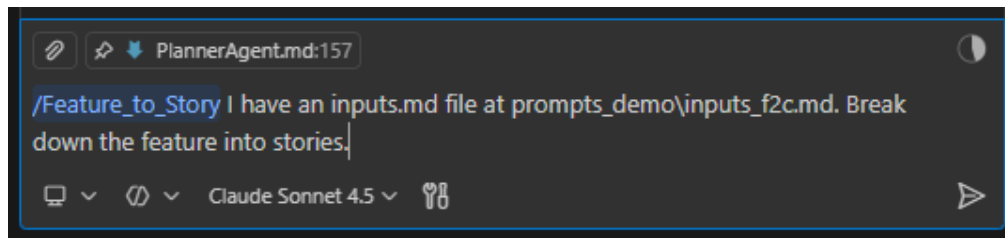
### Impact Points
- API backend logic automation
- Database configuration automation
- Sub-phase configuration creation
- Role-based access control (GBAdmin, Power User)
- Build/release cycle independence

### Additional Labels
Story, 10-R1

### Assignees
<!-- None specified -->
```

Step 2: Trigger in VSCode Copilot Chat

/Feature_to_Story I have an inputs.md file at /path/to/inputs.md. Break down the feature into stories.



Step 3: Review and Confirm

The agent analyzes the feature and asks for confirmation before creating stories.



- ✓ Reviewed inputs f2c.md

Input Validation Summary:

Required Fields:

- ✓ Feature Issue Reference: <https://github.com/boschdevcloud/MiDAS/MiDAS-Platform/issues/3786>
- ✓ Repository Owner: MiDAS
- ✓ Repository Name: MiDAS-Platform

Optional Fields:

- **Impact Points:**
 - API backend logic automation
 - Database configuration automation
 - Sub-phase configuration creation
 - Role-based access control (GBAdmin, Power User)
 - Build/release cycle independence
- **Additional Labels:** Story, 10-R1
- **Assignees:** None specified

🔗 Run Create or update issue. – github (MCP Server)

Create a new or update an existing issue in a GitHub repository.

Input

```
{ "body": "***Related to:** #3786\n\n## Overview\nThis story focuses on implementing the core RESTful API endpoints
```

[See more](#)

Note that MCP servers or malicious conversation content may attempt to misuse 'Code' through tools.

Allow | Skip

🔧 Run Create or update issue. – github (MCP Server)

Create a new or update an existing issue in a GitHub repository.

Input

[See more](#)

Note that MCP servers or malicious conversation content may attempt to misuse 'Code' through tools.

Allow | Skip

🔍 Starting: Create story issues on GitHub (4/5)

⌂ Loading...

- 

Created Stories:

- #3787: API Endpoints for Use Case Configuration Management**
 - Implements core REST API endpoints (CREATE, READ, UPDATE) in SS6
 - Creates router, data models, and service layer
 - Integrates with `config_schema` database tables
- #3788: Role-Based Access Control Integration for Configuration APIs**
 - Implements RBAC using SS12 authorization system
 - Restricts access to GBAAdmin and Power users
 - Adds authorization decorators and permission definitions
- #3789: Configuration Schema Design and Validation Framework**
 - Defines JSON schemas for use case configurations
 - Implements runtime validation with detailed error messages
 - Stores schemas in database with versioning
- #3790: Sub-Phase Configuration Management System**
 - Enables dynamic sub-phase creation via API
 - Manages phase-to-use-case relationships
 - Implements sequence ordering and validation
- #3791: Existing API Enhancement and Database Integration**
 - Enhances existing `/services/api/v1/usecase-onboard` endpoint
 - Consumes configurations from database instead of hardcoded values
 - Maintains backward compatibility and Git workflow
- #3792: Documentation, Testing, and Monitoring Implementation**
 - Creates comprehensive user guide and API documentation
 - Implements test suite achieving >95% coverage
 - Sets up monitoring, alerting, and audit logging

Impact Points Addressed:

- ✓ **API backend logic automation** - Stories #1, #5
- ✓ **Database configuration automation** - Stories #3, #5
- ✓ **Sub-phase configuration creation** - Story #4
- ✓ **Role-based access control (GBAdmin, Power User)** - Story #2
- ✓ **Build/release cycle independence** - Stories #5, #6

Parent Feature Updated:

The parent feature issue #3786 now includes a **Subtasks** section with checkboxes linking to all 6 story issues, making it easy to track progress and maintain traceability between the feature and its implementation stories.

All stories are ready for developer assignment and implementation! 🚀

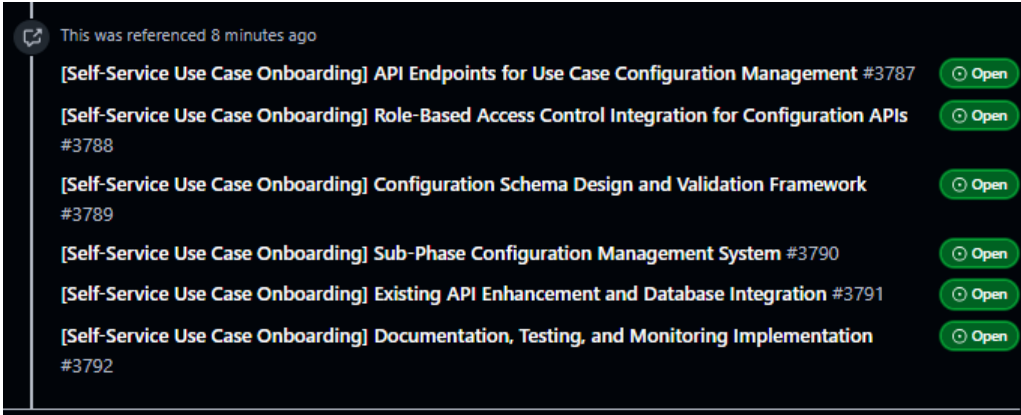
Step 4: Stories Created

Multiple story issues are created and linked to the parent feature.

Subtasks

- ☐ [#3787: API Endpoints for Use Case Configuration Management](#)
- ☐ [#3788: Role-Based Access Control Integration for Configuration APIs](#)
- ☐ [#3789: Configuration Schema Design and Validation Framework](#)
- ☐ [#3790: Sub-Phase Configuration Management System](#)
- ☐ [#3791: Existing API Enhancement and Database Integration](#)
- ☐ [#3792: Documentation, Testing, and Monitoring Implementation](#)





Quick Reference

Agent	Input	Output
Create Feature	Feature description + repo context	GitHub feature issue
Feature to Story	Feature issue URL/number	Multiple story issues

Tips

- Keep feature descriptions clear and value-focused
- Provide documentation paths for better context
- Use impact points to focus story creation
- Review generated content before confirming

Summary

Use these agents directly in VSCode Copilot Chat to automate GitHub issue creation and ensure consistent planning across your team.

